

lab8

要求：在原先os-part4仓库中完成信号量相关调用 `test_ipc_producer_consumer`
`test_ipc_philosopher`

▼ 前置操作

- 运行这个指令，切换到part8的分支上

```
git checkout part8-sem
```

- 进入这个仓库下，如果直接运行 `make run_test`，会提示缺少系统调用，在我们注册好四个调用后，用 `return 0` 占位，这个时候再运行似乎是可以完美通过两个测试点的。

```
extern uint64 sys_sem_p(void);
extern uint64 sys_sem_v(void);
extern uint64 sys_sem_create(void);
extern uint64 sys_sem_destroy(void);
```

- 不过参考助教提供的主要步骤，我们可以只在这一个分支下完成我们的所有任务，也就是说实现了上述这四个系统调用后，两个样例均可直接通过。其中 `create` 和 `destroy` 分别管理着信号量的生命周期。

▼ sys_sem_create

- 在 xv6-riscv 操作系统中，为协调多个进程对共享资源的并发访问，防止出现数据竞争和不一致的状态，其内核已经提供了**自旋锁**。自旋锁为**忙等待**的方式，但阻塞cpu不利于单核系统，因此需要我们提供 `sem` 去完成更灵活的操作。

这里的 `sem`，应以自旋锁或睡眠锁为基础，添加计数功能。

- ▼ 我们看一下xv6自己的自旋锁是什么情况。

```
// Mutual exclusion lock.
struct spinlock {
    uint locked;           // Is the lock held?

    // For debugging:
    char *name;            // Name of lock.
};
```

```

    struct cpu *cpu;    // The cpu holding the lock.
};

// Initialize a spinlock
void initlock(struct spinlock*, char*);

// Acquire the spinlock
// Must be used with release()
void acquire(struct spinlock*);

// Release the spinlock
// Must be used with acquire()
void release(struct spinlock*);

// Check whether this cpu is holding the lock
// Interrupts must be off
int holding(struct spinlock*);

```

实际上，这里的自旋锁非常简单，它只支持互斥的操作，因此其结构中只包含了一个 `locked` 来判断是否处于等待阶段，在这个逻辑下，它只需要由 `acquire` 来控制加锁，并阻塞其他进程，`release` 来控制解锁。

```

// Acquire the lock.
// Loops (spins) until the lock is acquired.
void
acquire(struct spinlock *lk)
{
    push_off(); // disable interrupts to avoid deadloc
    k.
    if(holding(lk))
        panic("acquire");

    // On RISC-V, sync_lock_test_and_set turns into an
    atomic swap:
    //   a5 = 1
    //   s1 = &lk->locked
    //   amoswap.w.aq a5, a5, (s1)
    //  你可以看到下面这一条，只要lk不是解锁状态，就死循环等待

```

```

while(__sync_lock_test_and_set(&lk->locked, 1) !=
0)
    ;

    // Tell the C compiler and the processor to not move loads or stores
    // past this point, to ensure that the critical section's memory
    // references happen strictly after the lock is acquired.
    // On RISC-V, this emits a fence instruction.
    __sync_synchronize();

    // Record info about lock acquisition for holding() and debugging.
    lk->cpu = mycpu();
}

```

- 我们可以依托现有的自旋锁来完成我们信号量的构造，事实上，不同于自旋锁，我们的信号量 `sem` 完全由计数器 `value` 这个判断指标来决定是否阻塞该进程。
 - `NSEM` 表示这个系统下最多可以同时支持的不同信号数量。

```

#define NSEM 128

struct sem
{
    int value;           // 信号量的值
    struct spinlock lock; // 保护信号量的自旋锁
    int valid;           // 信号量是否正在被使用
};

```

```

struct
{
    struct sem sems[NSEM]; // 信号量数组
    struct spinlock lock;   // 保护信号量数组的自旋锁
} sem_pool;

```

- 参考之前lab6有关交换区的结构体定义流程，我们也需要一个全局初始化的函数 `void seminit();`，并将其放在 `main.c` 中即可。

```
void seminit()
{
    initlock(&sem_pool.lock, "sem_pool_lock");
    int i;
    for (i = 0; i < NSEM; i++)
    {
        initlock(&sem_pool.sems[i].lock, "semaphore");
        sem_pool.sems[i].value = 0;
        sem_pool.sems[i].valid = 0;
    }
}
```

- 参考测试样例中 `sem_create` 的用法，我们知道它会向内核传递一个参数，用于初始化信号量的计数器。考虑到 `sem_p` 和 `sem_v` 中传递的参数为一个整数，并且在测试样例中，锁的数据类型也是一个int，我们知道这里的create应该起到了一个分配 `sem_pool` 中信号量的作用，需要返回一个索引值（或者说信号）。

```
int sem_create(int initial_value)
{
    acquire(&sem_pool.lock);
    for (int i = 0; i < NSEM; i++)
    {
        if (sem_pool.sems[i].valid == 0)
        {
            sem_pool.sems[i].valid = 1;
            sem_pool.sems[i].value = initial_value;
            release(&sem_pool.lock);
            return i;
        }
    }
    release(&sem_pool.lock);
    return -1; // No available semaphore
}
```

- 在 `sysproc.c` 中，包装好这个函数即可。

```
uint64
sys_sem_create(void)
{
    int initial_value;
    if (argint(0, &initial_value) < 0)
    {
        return -1;
    }
    return sem_create(initial_value);
}
```

▼ sys_sem_destroy

- 如果要回收一个信号量，我们需要检查是否有其他进程在等待这个信号量而睡眠，假设我们什么都不做直接回收，那么等待中的进程切换回来执行下一次等待时，就会试图获取一个完全不存在的内容。
- 因此我们应该在回收信号量前将所有进程都唤醒。在 `proc.c` 中，我们找到 `wakeup` 的定义，它可以将所有 `p->chan` 的睡眠进程全部唤醒。

```
void
wakeup(void *chan)
{
    struct proc *p;

    for(p = proc; p < &proc[NPROC]; p++) {
        acquire(&p->lock);
        if(p->state == SLEEPING && p->chan == chan) {
            p->state = RUNNABLE;
        }
        release(&p->lock);
    }
}
```

不过这里的 `chan` 是什么？在系统中，它是一个没有明确类型的指针变量，我们将其理解为 `struct proc` 下的一个成员变量，用于记录该进程 `SLEEP` 状态的来源即可。

```

void
sleep(void *chan, struct spinlock *lk)
{
    struct proc *p = myproc();

    if(lk != &p->lock){ //DOC: sleeplock0
        acquire(&p->lock); //DOC: sleeplock1
        release(lk);
    }

    // Go to sleep.
    p->chan = chan;    // 在这一步，sleep将该进程的chan设置
    // 为传入的第一个参数。
    p->state = SLEEPING;

    sched();

    // Tidy up.
    p->chan = 0;

    // Reacquire original lock.
    if(lk != &p->lock){
        release(&p->lock);
        acquire(lk);
    }
}

```

- 我们在之后的信号量pv操作中，会把信号量作为睡眠来源传入 `sleep`，所以我们也只需要对 `wakeup` 传入同样参数即可。

```

int sem_destroy(int sem_id)
{
    if (sem_id < 0 || sem_id >= NSEM)
    {
        return -1; // Invalid semaphore ID
    }
    acquire(&sem_pool.lock);
    struct sem *s = &sem_pool.sems[sem_id];
}

```

```

    acquire(&s->lock);
    if (s->valid == 0)
    {
        release(&s->lock);
        release(&sem_pool.lock);
        return -1; // Semaphore not in use
    }
    sem_pool.sems[sem_id].valid = 0;
    wakeup(s); // Wake up any waiting processes
    release(&s->lock);
    release(&sem_pool.lock);
    return 0; // Success
}

```

- 在 `sysproc.c` 中同样添加一个包装即可。

▼ sys_sem_p

- 接下来是P操作，它在课程中用于减少信号量的计数，当信号量计数归零时，我们不使用自旋锁的acquire机制忙等待，要主动调用 `sleep` 让出cpu。不用 `yield` 的原因是我们配套使用 `sleep` / `wakeup` 来管理信号量。

```

int sem_p(int sem_id)
{
    if (sem_id < 0 || sem_id >= NSEM)
    {
        return -1; // Invalid semaphore ID
    }
    struct sem *s = &sem_pool.sems[sem_id];
    acquire(&s->lock);
    while (s->value <= 0)
    {
        sleep(s, &s->lock); // Sleep until the semaphore
        is available
    }
    s->value--;
    release(&s->lock);
    return 0;
}

```

- 同样在 `sysproc.c` 中添加包装。

▼ `sys_sem_v`

- 这个调用更加简单，别忘了最后在 `sysproc.c` 中添加包装。

```
int sem_v(int sem_id)
{
    if (sem_id < 0 || sem_id >= NSEM)
    {
        return -1; // Invalid semaphore ID
    }
    struct sem *s = &sem_pool.sems[sem_id];
    acquire(&s->lock);
    s->value++;
    // 这一步一定要有!
    wakeup(s);

    release(&s->lock);
    return 0;
}
```

为什么还要添加 `wakeup(s)` 这一步？如果执行了 `s->value++`，下一次切换到另一个阻塞在 `s` 的进程时，`while` 检查 `s->value` 不就不会持续 `sleep` 了吗？

其实并非，在 `sleep` 之后，进程的状态将永远被设置为 `SLEEP`，不能被调度，因此其它等待该信号量的进程永远无法被唤醒。